

MLP

March 2026

6.1 Theory

a) Vanishing Gradient Problem

The **vanishing gradient problem** occurs in deep neural networks when the gradients of the loss with respect to weights in earlier layers become extremely small during backpropagation. As a result, the weights in the initial layers update very slowly, making it difficult for the network to learn useful representations.

Consider the sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Its derivative is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Since $0 < \sigma(z) < 1$, we have

$$0 < \sigma'(z) \leq 0.25$$

The maximum value occurs at $z = 0$:

$$\sigma'(0) = 0.25$$

Consider a neural network with L layers. During backpropagation, using the chain rule, the gradient of the loss L with respect to the weights in the first layer $W^{(1)}$ is

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial a^{(L)}} \frac{\partial a^{(L)}}{\partial a^{(L-1)}} \frac{\partial a^{(L-1)}}{\partial a^{(L-2)}} \cdots \frac{\partial a^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial W^{(1)}}$$

Each term $\frac{\partial a^{(l)}}{\partial a^{(l-1)}}$ contains the derivative of the sigmoid function. Therefore the gradient includes a product of sigmoid derivatives:

$$\prod_{l=1}^L \sigma'(z^{(l)})$$

Since $\sigma'(z^{(l)}) \leq 0.25$, the gradient magnitude is bounded by

$$(0.25)^L$$

As the depth L increases, this value decreases exponentially and approaches zero. Consequently, gradients flowing to earlier layers become extremely small, preventing effective learning. This phenomenon is known as the **vanishing gradient problem**.

b) Chain Rule in a Random Graph Model with Dropped Units

Suppose hidden units are randomly dropped with probability 0.5, similar to dropout. Let

$$m_i \sim \text{Bernoulli}(0.5)$$

be a binary mask indicating whether neuron i is active. The activation of neuron i becomes

$$a_i = m_i f(z_i)$$

During backpropagation, applying the chain rule gives

$$\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_i}$$

Since $a_i = m_i f(z_i)$, we obtain

$$\frac{\partial a_i}{\partial z_i} = m_i f'(z_i)$$

Thus,

$$\frac{\partial L}{\partial z_i} = m_i f'(z_i) \frac{\partial L}{\partial a_i}$$

If $m_i = 0$, the neuron is dropped and

$$\frac{\partial L}{\partial z_i} = 0$$

so no gradient flows through that neuron. If $m_i = 1$, gradients propagate normally.

Therefore, the **chain rule itself does not change**, but the computational graph becomes a randomly sampled subgraph where gradients flow only through active neurons, while dropped neurons receive zero gradient

Mathematical Derivations

1. Activation Functions

1.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Proof:

$$\frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = \sigma(x)(1 - \sigma(x))$$

—

1.2 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivative:

$$\tanh'(x) = 1 - \tanh^2(x)$$

Proof:

$$\frac{d}{dx} \tanh(x) = \frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2} = \frac{4}{(e^x + e^{-x})^2} = 1 - \tanh^2(x)$$

—

1.3 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Derivative:

$$\text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

—

1.4 Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

Derivative:

$$\text{LeakyReLU}'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$

where $\alpha = 0.01$.

—

2. Loss Functions

2.1 Mean Squared Error (MSE)

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Derivative:

$$\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i)$$

Proof:

$$\frac{\partial}{\partial \hat{y}_i} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right) = \frac{1}{n} \cdot 2(y_i - \hat{y}_i)(-1) = \frac{2}{n} (\hat{y}_i - y_i)$$

—

2.2 Mean Absolute Error (MAE)

$$\mathcal{L}_{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Derivative:

$$\frac{\partial \mathcal{L}_{MAE}}{\partial \hat{y}_i} = \text{sign}(\hat{y}_i - y_i)$$

Proof:

$$\frac{\partial}{\partial \hat{y}_i} |y_i - \hat{y}_i| = \frac{\hat{y}_i - y_i}{|y_i - \hat{y}_i|} = \text{sign}(\hat{y}_i - y_i)$$

Derivative is constant (± 1) regardless of error magnitude, which is why MAE converges slower than MSE.

—

2.3 Huber Loss

$$\mathcal{L}_\delta = \frac{1}{n} \sum_{i=1}^n \begin{cases} \frac{1}{2}(y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| \leq \delta \\ \delta (|y_i - \hat{y}_i| - \frac{\delta}{2}) & |y_i - \hat{y}_i| > \delta \end{cases}$$

Derivative:

$$\frac{\partial \mathcal{L}_\delta}{\partial \hat{y}_i} = \begin{cases} \hat{y}_i - y_i & |\hat{y}_i - y_i| \leq \delta \\ \delta \cdot \text{sign}(\hat{y}_i - y_i) & |\hat{y}_i - y_i| > \delta \end{cases}$$

3. Optimizers

3.1 SGD

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

where η is the learning rate.

3.2 Momentum

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

where $\beta = 0.9$ typically.

3.3 Nesterov Accelerated Gradient

$$v_t = \beta v_{t-1} - \eta \nabla \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \beta v_{t-1} + (1 + \beta) v_t$$

Gradient evaluated at lookahead position $\theta_t + \beta v_{t-1}$.

3.4 AdaGrad

$$G_t = G_{t-1} + g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

3.5 RMSProp

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

where $\beta = 0.9$, $\epsilon = 10^{-8}$.

—

3.6 Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

—

3.7 Muon

$$m_t = \beta m_{t-1} + (1 - \beta)g_t$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\|m_t\| + \epsilon} m_t$$

—

4. Weight Initialization

4.1 Xavier / Glorot Initialization

$$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$$

Variance propagation:

$$\text{Var}(z) = n_{in} \cdot \text{Var}(W) \cdot \text{Var}(x)$$

To keep $\text{Var}(z) = \text{Var}(x)$:

$$n_{in} \text{Var}(W) = 1$$

$$\text{Var}(W) = \frac{1}{n_{in}}$$

4.2 He Initialization

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

ReLU keeps half activations:

$$\text{Var}(z) = \frac{n_{in}}{2} \text{Var}(W) \text{Var}(x)$$

To keep $\text{Var}(z) = \text{Var}(x)$:

$$\frac{n_{in}}{2} \text{Var}(W) = 1$$

$$\text{Var}(W) = \frac{2}{n_{in}}$$